

UTS

PENGOLAHAN CITRA DIGITAL



NAMA : Sharira Maharani

NIM : 202431001

KELAS : E

DOSEN : Rizqia Cahyaningtyas, ST., M.Kom

NO.PC : 9

ASISTEN : 1. Joshua Indra Pratama

2. Davina Najwa Ermawan

3. Izzat Islami Kagapi

4. Sakura Amastasya Salsabila Setiyanto

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2025/2026

DAFTAR ISI

DAFTAR ISI.....	2
BAB I.....	3
PENDAHULUAN.....	3
1.1 Rumusan Masalah	3
1.2 Tujuan Masalah	3
1.3 Manfaat Masalah	3
BAB II.....	4
LANDASAN TEORI	4
2.1 Pengolahan Citra Digital	4
2.2 Model Warna RGB (Red, Green, Blue).....	4
2.3 Konversi Citra RGB ke Grayscale.....	5
2.3.1 Metode Luminansi (<i>Luminosity</i>).....	5
2.3.2 Metode Rata-Rata (<i>Average Method</i>)	5
2.4 Operasi Piksel pada Pengolahan Citra Digital	6
2.4.1 Operasi Kecerahan (<i>Brightness</i>)	6
2.4.2 Operasi Kontras (<i>Contrast</i>).....	6
2.4.3 Operasi Negatif Citra.....	6
2.5 Histogram Citra Digital	7
2.5.1 Pemerataan Histogram (<i>Histogram Equalization</i>)	7
2.6 Thresholding (Pengambangan).....	7
2.6.1 <i>Global Thresholding</i>	8
2.6.2 Metode Otsu	8
2.7 Citra Grayscale.....	8
2.8 Segmentasi Citra dengan Binary Thresholding	8
BAB III	9
HASIL.....	9
3.1 Asset Gambar	9
3.2 Ekstraksi Warna, Konversi Warna dan Thresholding.....	10
3.3 Operasi Piksel	20
BAB IV	28
PENUTUP.....	28
DAFTAR PUSTAKA.....	29

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

1. Bagaimana cara melakukan ekstraksi kanal warna (Red, Green, Blue) secara akurat dari sebuah citra digital menggunakan teknik *splitting array*?
2. Bagaimana pengaruh nilai ambang batas (*threshold*) yang berbeda terhadap keberhasilan deteksi objek tulisan tangan pada masing-masing kanal warna?
3. Bagaimana cara mengoptimalkan kualitas visual citra profil melalui manipulasi parameter *Alpha* (kontras) dan *Beta* (kecerahan) agar objek utama lebih menonjol dibandingkan latar belakang?
4. Bagaimana efektivitas penggunaan operasi logika *Bitwise* dan pengurangan citra (*subtraction*) dalam menyatukan kembali informasi warna yang telah terpisah menjadi satu citra biner yang bersih?

1.2 Tujuan Masalah

1. Berhasil memisahkan komponen warna primer (RGB) dari citra asli untuk menganalisis karakteristik unik setiap tinta warna melalui distribusi histogram.
2. Menentukan nilai ambang batas yang paling optimal untuk setiap kanal guna memisahkan objek tulisan nama dari latar belakang kertas tanpa merusak detail coretan tinta.
3. Menerapkan teknik perbaikan citra melalui transformasi linear manual guna meningkatkan kontras pada area yang gelap dan mencerahkan area yang redup pada foto profil.
4. Menghasilkan citra biner yang menunjukkan hasil deteksi warna secara utuh dan profesional sebagai bentuk verifikasi keberhasilan pengolahan citra digital.

1.3 Manfaat Masalah

1. Memahami secara mendalam cara kerja manipulasi piksel demi piksel dan transformasi ruang warna menggunakan library OpenCV dan NumPy.
2. Meningkatkan kemampuan dalam membaca dan menginterpretasikan data statistik histogram untuk mengambil keputusan teknis, seperti penentuan nilai *threshold* yang tepat.
3. Memberikan keterampilan dalam memperbaiki kualitas visual sebuah foto secara terukur, yang sangat berguna untuk proses identifikasi objek atau *computer vision* di masa depan.
4. Menghasilkan dokumentasi pengolahan citra yang sistematis, di mana informasi penting (seperti nama dan profil) menjadi lebih jelas dan mudah diidentifikasi dibandingkan data mentahnya.

BAB II

LANDASAN TEORI

2.1 Pengolahan Citra Digital

Pengolahan citra digital (*digital image processing*) merupakan serangkaian operasi komputasi yang diterapkan pada citra digital untuk meningkatkan kualitas visual, mengekstrak informasi, atau mentransformasi citra ke dalam representasi lain yang lebih berguna. Citra digital dapat didefinisikan sebagai fungsi dua dimensi $f(x, y)$ di mana x dan y merupakan koordinat spasial, sedangkan nilai fungsi di setiap pasang koordinat disebut intensitas atau nilai keabuan (*gray level*) pada titik tersebut.

Citra digital tersusun dari elemen-elemen terkecil yang disebut piksel (*picture element*). Setiap piksel memiliki nilai intensitas yang berkisar antara 0 hingga 255 untuk citra dengan kedalaman 8 bit. Nilai 0 merepresentasikan warna hitam penuh, sedangkan nilai 255 merepresentasikan warna putih penuh. Dalam citra berwarna, setiap piksel umumnya dinyatakan oleh tiga komponen warna, yaitu merah (*Red*), hijau (*Green*), dan biru (*Blue*), yang dikenal sebagai model warna RGB.

Tahapan umum dalam pengolahan citra digital meliputi akuisisi citra, pra-pemrosesan (*preprocessing*), segmentasi, representasi dan deskripsi, serta pengenalan dan interpretasi. Setiap tahap memiliki peran penting dalam menghasilkan informasi yang akurat dari citra yang diperoleh. Proses pra-pemrosesan seringkali menjadi tahap kritis karena kualitas citra pada tahap awal sangat menentukan kualitas hasil akhir analisis.

2.2 Model Warna RGB (Red, Green, Blue)

Model warna RGB merupakan model warna aditif yang paling umum digunakan dalam pengolahan citra digital. Pada model ini, setiap warna direpresentasikan sebagai kombinasi dari tiga komponen warna primer, yaitu merah (Red/R), hijau (Green/G), dan biru (Blue/B). Ketiga komponen tersebut masing-masing memiliki rentang nilai intensitas dari 0 hingga 255 pada sistem 8-bit, sehingga total kombinasi warna yang dapat direpresentasikan adalah $256^3 = 16.777.216$ warna yang berbeda.

Setiap piksel pada citra berwarna dinyatakan sebagai vektor tiga dimensi RGB(R, G, B). Sebagai contoh, warna putih direpresentasikan sebagai RGB(255, 255, 255), warna hitam sebagai RGB(0, 0, 0), warna merah murni sebagai RGB(255, 0, 0), warna hijau murni sebagai RGB(0, 255, 0), dan warna biru murni sebagai RGB(0, 0, 255). Pada citra berwarna format RGB, sebuah citra berukuran $M \times N$ piksel membutuhkan tiga matriks berukuran $M \times N$ untuk menyimpan nilai komponen R, G, dan B secara terpisah.

Representasi matematis sebuah citra berwarna RGB dapat dinyatakan sebagai berikut:

$$f(x, y) = [R(x, y), G(x, y), B(x, y)]$$

di mana $R(x,y)$, $G(x,y)$, dan $B(x,y)$ masing-masing merepresentasikan nilai intensitas komponen merah, hijau, dan biru pada koordinat piksel (x, y) . Model RGB menjadi dasar bagi berbagai operasi pengolahan citra, termasuk konversi ke ruang warna lain seperti grayscale, HSV, dan YcbCr.

2.3 Konversi Citra RGB ke Grayscale

Konversi citra berwarna RGB menjadi citra grayscale merupakan salah satu operasi pra-pemrosesan yang paling fundamental dalam pengolahan citra digital. Tujuan utama konversi ini adalah untuk menyederhanakan representasi citra dari tiga kanal warna menjadi satu kanal intensitas, sehingga mengurangi kompleksitas komputasi sekaligus mempertahankan informasi struktural penting dari citra asli. Proses ini menjadi tahap awal yang lazim digunakan sebelum menerapkan algoritma lebih lanjut seperti deteksi tepi, segmentasi, dan klasifikasi.

Terdapat beberapa metode yang dapat digunakan untuk melakukan konversi citra RGB ke grayscale, antara lain metode rata-rata (*average method*), metode nilai tengah (*midrange method*), dan metode luminansi (*luminance/luminosity method*). Di antara metode-metode tersebut, metode luminansi dianggap yang paling akurat secara perseptual karena mempertimbangkan sensitivitas mata manusia yang berbeda-beda terhadap setiap komponen warna.

2.3.1 Metode Luminansi (*Luminosity*)

Metode luminansi merupakan pendekatan konversi grayscale yang paling banyak digunakan dan direkomendasikan oleh standar ITU-R BT.601. Metode ini menggunakan bobot yang berbeda untuk setiap komponen warna RGB berdasarkan sensitivitas fotoreseptor pada mata manusia. Mata manusia lebih sensitif terhadap warna hijau, diikuti merah, kemudian biru. Rumus konversi luminansi dinyatakan sebagai berikut:

$$Gray = 0.299 \times R + 0.587 \times G + 0.114 \times B$$

Di dalam rumus tersebut, Gray merupakan nilai intensitas piksel pada citra grayscale hasil konversi, sedangkan R, G, dan B masing-masing menyatakan nilai intensitas kanal merah, hijau, dan biru dari piksel citra RGB asli. Koefisien 0.299, 0.587, dan 0.114 dalam rumus tersebut tidak ditetapkan secara sembarangan, melainkan didasarkan pada kontribusi relatif masing-masing kanal warna terhadap persepsi kecerahan oleh mata manusia. Koefisien komponen hijau (0.587) paling besar karena mata manusia paling sensitif terhadap warna hijau, diikuti merah (0.299), dan biru (0.114).

2.3.2 Metode Rata-Rata (*Average Method*)

Selain metode luminansi, metode rata-rata sederhana juga dapat digunakan untuk mengkonversi citra RGB ke grayscale. Metode ini menghitung nilai rata-rata aritmatika dari ketiga komponen warna RGB:

$$Gray = (R + G + B) / 3$$

Meskipun lebih mudah dihitung, metode rata-rata kurang akurat secara perseptual dibandingkan metode luminansi karena memberikan bobot yang sama untuk semua komponen warna tanpa mempertimbangkan sensitivitas mata manusia.

2.4 Operasi Pixel pada Pengolahan Citra Digital

Operasi pixel merupakan operasi dasar dalam pengolahan citra digital yang bekerja pada level pixel individual. Operasi ini mengubah nilai intensitas setiap pixel secara independen berdasarkan fungsi transformasi tertentu tanpa mempertimbangkan pixel-pixel tetangganya. Karena kesederhanaannya, operasi pixel sangat efisien secara komputasi dan sering digunakan sebagai tahap awal dalam alur pemrosesan citra.

Secara matematis, operasi pixel dapat dinyatakan sebagai:

$$S = T(r)$$

di mana r adalah nilai intensitas pixel pada citra masukan, s adalah nilai intensitas pixel pada citra keluaran, dan T adalah fungsi transformasi yang diterapkan. Fungsi T dapat berupa penambahan konstanta, perkalian, pemotongan nilai (*clipping*), maupun fungsi nonlinear lainnya.

2.4.1 Operasi Kecerahan (*Brightness*)

Kecerahan (*brightness*) merupakan persepsi subjektif terhadap intensitas cahaya keseluruhan pada sebuah citra. Operasi penyesuaian kecerahan dilakukan dengan menambahkan atau mengurangi konstanta c pada setiap nilai pixel dalam citra. Operasi ini dapat dinyatakan secara matematis sebagai berikut:

$$s(x, y) = r(x, y) + c$$

Apabila c bernilai positif, maka kecerahan citra akan meningkat (citra menjadi lebih terang). Sebaliknya, apabila c bernilai negatif, kecerahan citra akan berkurang (citra menjadi lebih gelap). Nilai hasil operasi harus dibatasi (*clipped*) dalam rentang $[0, 255]$ untuk mencegah terjadinya overflow maupun underflow.

2.4.2 Operasi Kontras (*Contrast*)

Kontras (*contrast*) merupakan perbedaan nilai intensitas antara daerah terang dan gelap pada sebuah citra. Penyesuaian kontras dilakukan dengan mengalikan nilai setiap pixel dengan faktor pengali α . Secara matematis, operasi ini dapat ditulis:

$$s(x, y) = \alpha \times r(x, y)$$

Apabila $\alpha > 1$, rentang nilai intensitas pixel akan melebar sehingga kontras meningkat. Apabila $0 < \alpha < 1$, kontras akan menurun. Kedua operasi kecerahan dan kontras sering digabungkan dalam satu formula sebagai: $s = \alpha \times r + c$, di mana α mengontrol kontras dan c mengontrol kecerahan.

2.4.3 Operasi Negatif Citra

Operasi negatif citra menghasilkan citra negatif dengan cara membalik nilai intensitas setiap pixel. Untuk citra dengan kedalaman 8 bit, transformasi negatif dinyatakan sebagai:

$$s(x, y) = 255 - r(x, y)$$

Operasi ini berguna untuk memperjelas detail yang terdapat pada area gelap dari citra, seperti pada citra medis mamografi. Pixel yang semula bernilai 0 (hitam) akan menjadi 255 (putih), dan sebaliknya.

2.5 Histogram Citra Digital

Histogram citra merupakan representasi grafis yang menggambarkan distribusi frekuensi nilai intensitas piksel pada sebuah citra digital. Sumbu horizontal histogram merepresentasikan nilai intensitas (0–255 untuk citra 8-bit), sedangkan sumbu vertikal merepresentasikan jumlah atau proporsi piksel yang memiliki nilai intensitas tersebut. Histogram memberikan gambaran menyeluruh tentang karakteristik kecerahan dan kontras suatu citra tanpa memperlihatkan informasi spasial.

Secara matematis, histogram citra didefinisikan sebagai:

$$h(rk) = nk$$

di mana rk adalah nilai intensitas ke- k dan nk adalah jumlah piksel dalam citra yang memiliki nilai intensitas rk . Histogram yang dinormalisasi dapat dihitung dengan membagi jumlah piksel pada setiap level intensitas dengan total piksel dalam citra ($M \times N$):

$$p(rk) = nk / (M \times N)$$

Dari histogram citra, beberapa informasi penting dapat dibaca secara langsung. Histogram yang terkonsentrasi di area nilai rendah (gelap) mengindikasikan citra yang kurang cahaya (*underexposed*). Sebaliknya, histogram yang terkonsentrasi di area nilai tinggi (terang) mengindikasikan citra yang terlalu terang (*overexposed*). Histogram yang tersebar merata di seluruh rentang nilai intensitas mengindikasikan citra dengan kontras yang baik.

2.5.1 Pemerataan Histogram (*Histogram Equalization*)

Pemerataan histogram (*histogram equalization*) merupakan teknik peningkatan kontras citra yang bekerja dengan cara mendistribusikan ulang nilai intensitas piksel sehingga histogram citra menjadi lebih merata. Teknik ini secara otomatis meningkatkan kontras citra dengan memperluas distribusi intensitas piksel ke seluruh rentang yang tersedia. Fungsi transformasi pada histogram equalization dihitung menggunakan fungsi distribusi kumulatif (*cumulative distribution function/CDF*) dari histogram asli.

2.6 Thresholding (Pengambangan)

Thresholding atau pengambangan merupakan teknik segmentasi citra yang paling sederhana dan paling banyak digunakan. Proses ini mengubah citra grayscale menjadi citra biner dengan cara membandingkan nilai intensitas setiap piksel dengan suatu nilai ambang batas (threshold). Piksel-piksel yang nilai intensitasnya melampaui atau sama dengan nilai threshold akan diubah menjadi nilai tertentu (biasanya putih = 255), sedangkan piksel yang nilainya di bawah threshold akan diubah menjadi nilai lainnya (biasanya hitam = 0). Tujuan utama thresholding adalah memisahkan objek yang diminati dari latar belakangnya.

Secara matematis, transformasi thresholding dinyatakan sebagai:

$$g(x, y) = \begin{cases} 255, & \text{jika } f(x, y) \geq T \\ 0, & \text{jika } f(x, y) < T \end{cases}$$

di mana $f(x, y)$ adalah nilai intensitas piksel pada citra input, $g(x, y)$ adalah nilai piksel pada citra biner hasil thresholding, dan T adalah nilai threshold yang digunakan. Pemilihan nilai T yang tepat sangat menentukan kualitas hasil segmentasi. Nilai threshold dapat ditentukan secara manual (*manual thresholding*) atau secara otomatis menggunakan metode seperti Otsu's thresholding.

2.6.1 Global Thresholding

Global thresholding menggunakan satu nilai ambang yang sama untuk seluruh piksel dalam citra. Metode ini sederhana dan efisien, namun kurang optimal untuk citra dengan pencahayaan yang tidak merata. Nilai threshold umumnya ditentukan berdasarkan histogram citra, yaitu pada titik lembah (*valley*) di antara dua puncak histogram yang merepresentasikan objek dan latar belakang.

2.6.2 Metode Otsu

Metode Otsu merupakan salah satu algoritma global thresholding otomatis yang paling populer. Algoritma ini menentukan nilai threshold optimal dengan cara memaksimalkan variansi antara dua kelas (objek dan latar belakang) atau secara ekuivalen meminimalkan variansi dalam kelas (*intra-class variance*). Metode Otsu menganalisis histogram citra dan secara otomatis menemukan nilai threshold yang paling optimal untuk memisahkan dua kelompok piksel tersebut.

2.7 Citra Grayscale

Citra grayscale adalah jenis citra digital yang setiap pikselnya hanya memiliki satu nilai intensitas, yang merepresentasikan derajat keabuan mulai dari hitam hingga putih. Pada citra grayscale 8-bit, nilai intensitas berkisar antara 0 (hitam penuh) hingga 255 (putih penuh), menghasilkan 256 tingkat keabuan yang berbeda. Berbeda dengan citra berwarna yang membutuhkan tiga matriks untuk menyimpan komponen RGB, citra grayscale hanya membutuhkan satu matriks sehingga lebih hemat memori dan lebih cepat diproses.

Citra grayscale sering digunakan sebagai tahap pra-pemrosesan sebelum berbagai algoritma pengolahan citra diterapkan, antara lain deteksi tepi (*edge detection*), segmentasi objek, pengenalan pola, dan kompresi citra. Penggunaan citra grayscale menyederhanakan komputasi tanpa mengorbankan informasi struktural penting yang terkandung dalam citra.

2.8 Segmentasi Citra dengan Binary Thresholding

Segmentasi citra merupakan proses membagi citra digital menjadi beberapa bagian atau segmen yang bermakna untuk mempermudah analisis. Binary thresholding adalah teknik segmentasi yang menghasilkan citra biner dengan dua nilai intensitas saja, yaitu hitam (0) dan putih (255). Teknik ini sangat efektif untuk memisahkan objek dari latar belakang, terutama ketika terdapat perbedaan kontras yang jelas antara keduanya.

Penerapan binary thresholding telah terbukti efektif dalam berbagai kasus, di antaranya segmentasi naskah kuno untuk keperluan digitalisasi warisan budaya, identifikasi objek dalam citra medis, inspeksi kualitas produk dalam industri manufaktur, serta pendeteksian kematangan buah pada bidang pertanian. Keberhasilan segmentasi sangat bergantung pada ketepatan pemilihan nilai threshold yang digunakan.

BAB III

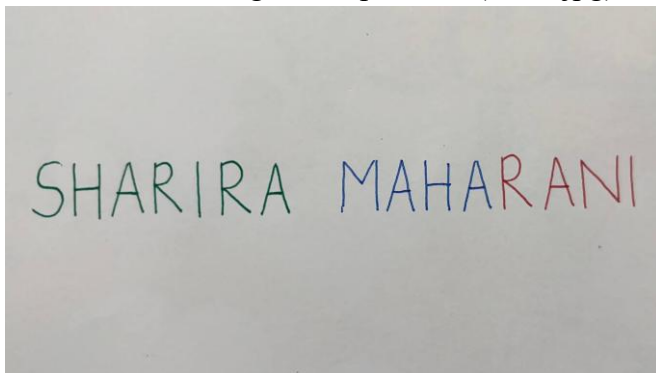
HASIL

Berikut adalah penjelasan lengkap mengenai alur pengerjaan pada UTS Praktikum Pengolahan Citra Digital. Proses dimulai dengan pemanggilan pustaka Python yang relevan, diikuti dengan pengolahan citra mentah berupa tulisan tangan berupa nama sesuai ketentuan soal. Setiap baris kode dirancang untuk mengubah data piksel dari format BGR asli menuju hasil akhir yang telah dioptimalkan secara visual. Penjelasan ini akan dibagi ke dalam beberapa bagian utama, mulai dari tahap ekstraksi warna, binerisasi dengan nilai ambang batas yang tepat, hingga analisis parameter Alpha dan Beta dalam meningkatkan kualitas citra.

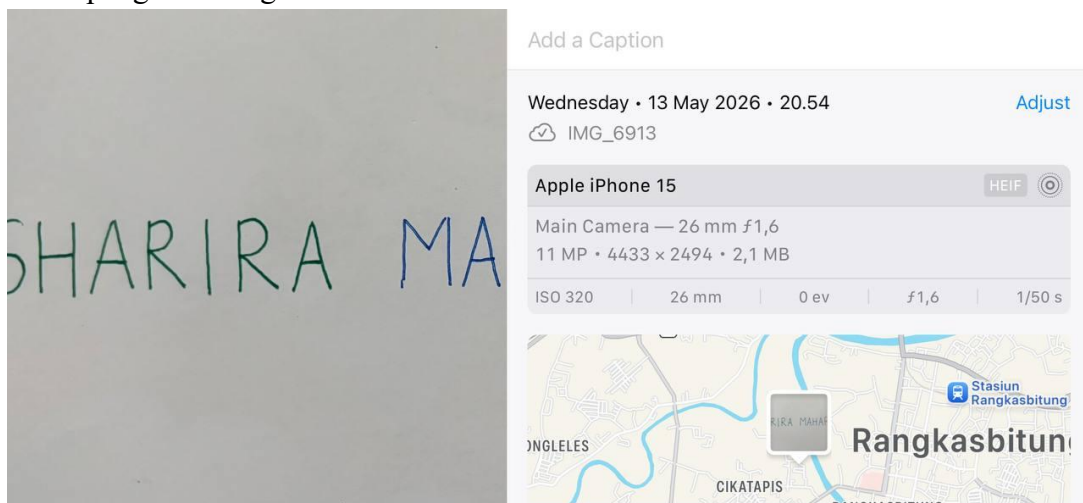
3.1 Asset Gambar

Terdapat dua gambar yang digunakan untuk pengolahan pada uts praktikum ini. Gambar diperoleh langsung dari kamera handphone yang diambil di dua lokasi berbeda.

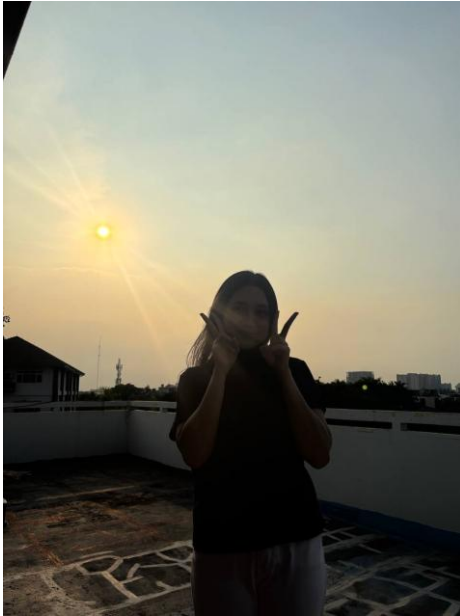
1. Gambar tulisan tangan berupa nama (nama.jpg)



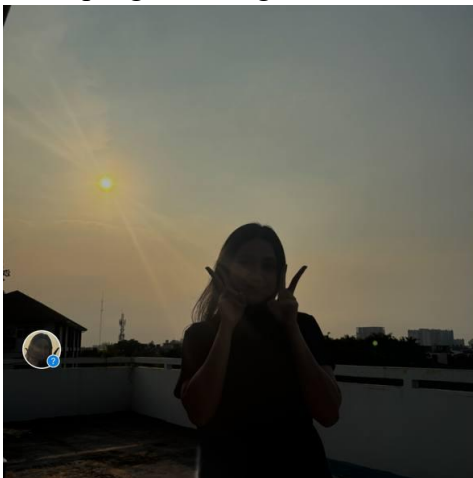
Bukti pengambilan gambar:



2. Gambar diri yang membelakangi sumber cahaya (myself.jpg)



Bukti pengambilan gambar:



Add a Caption

Monday • 11 May 2026 • 16.21

[Adjust](#)

IMG_6642

Apple iPhone 15

HEIF

Main Camera — 26 mm f1,6

24 MP • 4284 × 5712 • 1,5 MB

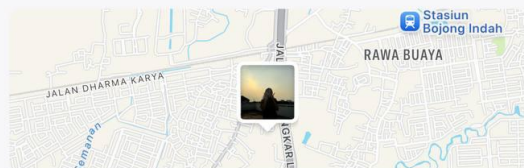
ISO 50

26 mm

-1,3 ev

f1,6

1/12195 s



3.2 Ekstraksi Warna, Konversi Warna dan Thresholding

- Import library

[1]:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

Pada bagian ini, kode bertujuan untuk memanggil library yang diperlukan dalam pengolahan citra. cv2 digunakan untuk membaca, memanipulasi kanal warna, dan melakukan transformasi citra. numpy digunakan untuk operasi matematika pada matriks citra, karena citra digital pada dasarnya adalah kumpulan angka dalam bentuk array. matplotlib.pyplot digunakan untuk menampilkan hasil gambar dan memvisualisasikan histogram secara rapi.

- Membaca gambar

[2]:

```
# Membaca file gambar
img = cv2.imread("nama.jpg")
```

Bagian cell kedua berfungsi untuk membaca file citra yang sudah ada di dalam folder dan disimpan dengan nama variabel `img`. Citra yang dibaca adalah potret tulisan tangan nama lengkap dengan tinta merah, hijau, dan biru di atas kertas putih. Secara default, OpenCV membaca gambar dalam format BGR (Blue-Green-Red), bukan RGB, sehingga urutan kanal warnanya akan berbeda saat diproses nanti.

- Menampilkan dimensi gambar

[3]:

```
# Melihat dimensi gambar
img.shape
```

[3]:

(720, 1280, 3)

Bagian ini digunakan untuk melihat dimensi dari gambar yang sudah dibaca tadi. Terlihat bahwa hasil dari kode ini adalah (720, 1280, 3), artinya gambar memiliki tinggi 720 piksel, lebar 1280 piksel, dan 3 kanal warna (B, G, R). Mengetahui dimensi ini tujuannya adalah untuk memastikan bahwa data yang diproses benar-benar berupa citra berwarna sebelum dilakukan ekstraksi kanal warna.

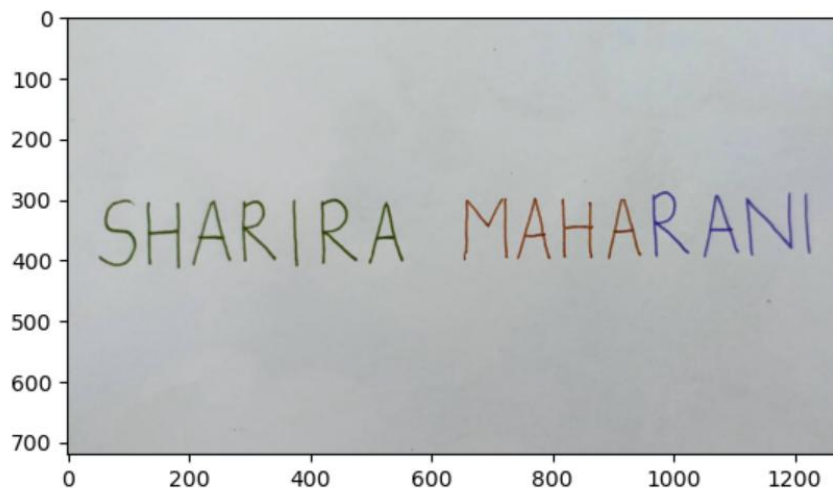
- Menampilkan gambar menggunakan library plt

[4]:

```
# Menampilkan gambar asli menggunakan plt
plt.imshow(img)
```

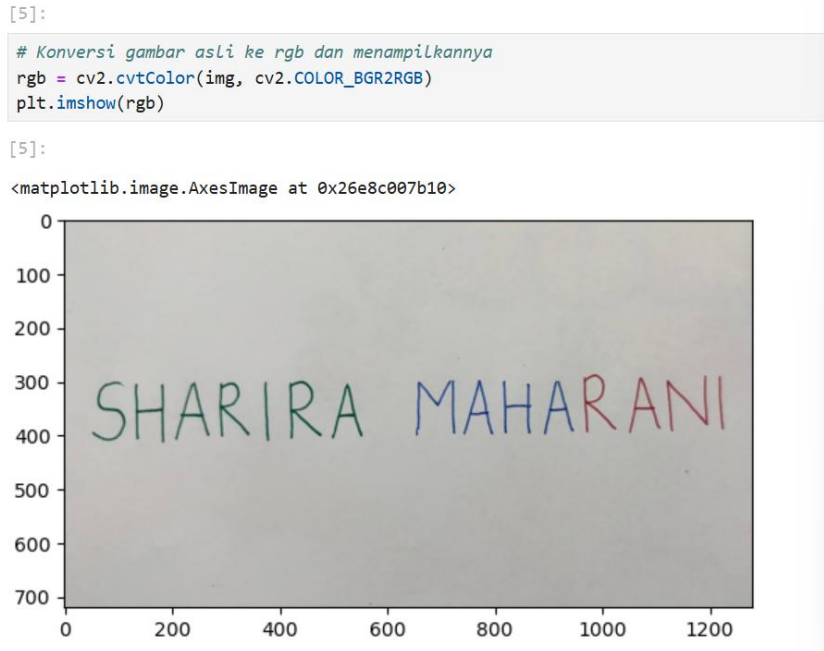
[4]:

<matplotlib.image.AxesImage at 0x26e8a1ef770>



Bagian ini digunakan untuk menampilkan gambar dengan library plt. Perintah `plt.imshow` akan memetakan setiap nilai piksel array ke dalam koordinat warna pada layar. Terlihat bahwa hasil gambar yang dihasilkan berbeda dengan gambar aslinya, hal ini dikarenakan sebelumnya gambar dibaca menggunakan `cv2.imread` (format BGR), maka saat ditampilkan langsung dengan `plt.imshow` (format RGB), warna pada gambar akan terlihat aneh atau terbalik. Tinta merah pada nama terlihat biru, dan tinta biru terlihat merah. Ini terjadi karena Matplotlib salah menginterpretasikan urutan kanal warna, yang secara teknis disebut sebagai channel swapping.

- Konversi warna pada gambar



Bagian ini berfungsi untuk mengubah skema warna gambar dari format BGR (Blue, Green, Red) menjadi format RGB (Red, Green, Blue) dengan menggunakan library cv2 dan disimpan ke dalam variabel bernama rgb. Jika tidak dilakukan konversi warna, data piksel pada variabel img akan tertukar. OpenCV akan membaca tulisan tinta merah di indeks kanal ke-2 (R), tapi jika ditampilkan tanpa konversi, matplotlib akan menganggap indeks ke-2 itu adalah Biru. Akibatnya, warna nama akan terlihat salah dan cenderung kebiruan. Selanjutnya hasil konversi tadi ditampilkan menggunakan plt.imshow(rgb).

- Ekstraksi kanal warna

```
[6]:
# Deklarasi variabel baru untuk setiap channel warna
r = rgb[:, :, 0]
g = rgb[:, :, 1]
b = rgb[:, :, 2]
```

Bagian ini merupakan proses splitting array, proses ini memisahkan matriks 3D menjadi tiga matriks 2D yang berbeda. Setiap matriks 2D ini sekarang hanya berisi nilai intensitas cahaya (0-255) untuk satu warna spesifik saja. Tanda : pertama berarti "ambil semua baris" (tinggi gambar). Tanda : kedua berarti "ambil semua kolom" (lebar gambar). Angka 0 berarti hanya mengambil indeks ke-n pada dimensi ketiga. Angka 0 menunjukkan urutan pertama dalam RGB, 1 berarti urutan kedua dan 2 berarti urutan ketiga. Variabel r menyimpan data dari kanal merah, g menyimpan kanal hijau dan b menyimpan kanal biru.

- Menampilkan hasil untuk setiap kanal warna

```
[7]:
# Menampilkan hasil setiap warna

fig, axs = plt.subplots(2,2, figsize=(15,8))
```

Keseluruhan dari bagian ini adalah visualisasi untuk membuktikan keberhasilan proses splitting array yang sebelumnya sudah dilakukan. Baris kode diatas digunakan untuk

menyiapkan kanvas dengan ukuran 15 x 8, subplots(2,2) berarti membuat grid yang terdiri dari 2 baris dan 2 kolom.

➤ Gambar asli

```
axs[0,0].imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
axs[0,0].set_title('Gambar Asli')
```

Hasil visualisasi:



Bagian pertama (baris 0, kolom 0) diisi dengan gambar asli yang sudah dikonversi ke RGB. Gambar diberi judul “Gambar Asli” untuk mempermudah pada saat mengidentifikasi gambar yang akan dibandingkan.

➤ Gambar deteksi merah

```
axs[0,1].imshow(r, cmap="gray")
axs[0,1].set_title('Deteksi Merah')
```

Hasil visualisasi:

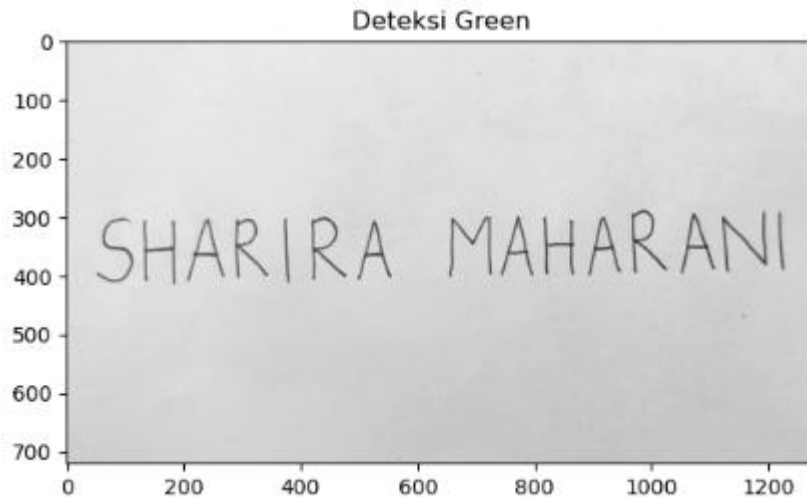


Bagian kedua diisi dengan variable r yaitu hasil splitting kanal merah. Baris kode `cmap="gray"` menampilkan gambar dalam skala abu – abu untuk melihat intensitasnya. Pada bagian ini, tulisan dengan tinta merah akan terlihat paling putih dibandingkan tulisan warna lain.

➤ Gambar deteksi hijau

```
axs[1,0].imshow(g, cmap="gray")
axs[1,0].set_title('Deteksi Green')
```

Hasil visualisasi:



Bagian ketiga ini diisi dengan variabel `g` yaitu hasil splitting kanal hijau. Penggunaan `cmap="gray"` tidak lagi menampilkan warna hijau secara fisik, melainkan menunjukkan intensitas cahaya hijau tersebut dalam skala keabuan.

➤ Gambar deteksi biru

```
axs[1,1].imshow(b, cmap="gray")
axs[1,1].set_title('Deteksi Biru')
```

Hasil visualisasi:



Bagian terakhir diisi dengan variabel `b` yaitu matriks kanal ketiga hasil dari splitting kanal biru. Sama seperti kanal lainnya, hasil yang ditampilkan adalah huruf dengan warna biru akan berwarna paling terang.

- Visualisasi histogram

[8]:

```
fig, axs = plt.subplots(4, 2, figsize=(15, 20))
```

Bagian pertama dilakukan pembuatan grid berukuran 15 x 20, dengan 4 baris dan 2 kolom menggunakan `plt.subplots`. Kolom pertama digunakan untuk menampilkan gambar, sedangkan kolom kedua untuk grafiknya.

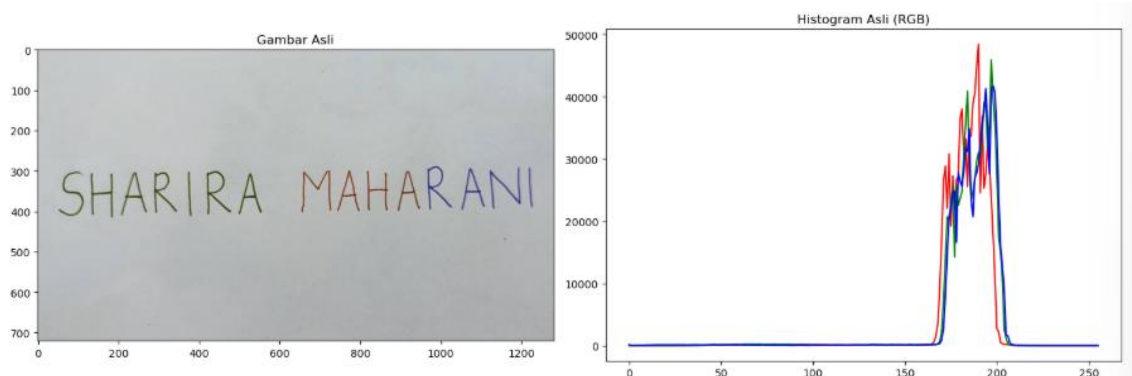
```
hist_asli_r = cv2.calcHist([img], [0], None, [256], [0, 256])
hist_asli_g = cv2.calcHist([img], [1], None, [256], [0, 256])
hist_asli_b = cv2.calcHist([img], [2], None, [256], [0, 256])
```

Variabel hist_asli_r/g/b berfungsi sebagai wadah penyimpanan data distribusi intensitas kanal merah, hijau dan biru dari citra asli. Fungsi cv2.calcHist digunakan untuk menghitung frekuensi kemunculan setiap nilai intensitas piksel (dari 0-255). Parameter [256] menentukan jumlah kotak yang akan dihitung, dan [0, 256] menentukan rentang nilai yang dihitung.

➤ Gambar Asli

```
axs[0, 0].imshow(img)
axs[0, 0].set_title('Gambar Asli')
axs[0, 1].plot(hist_asli_r, color='r')
axs[0, 1].plot(hist_asli_g, color='g')
axs[0, 1].plot(hist_asli_b, color='b')
axs[0, 1].set_title('Histogram Asli (RGB)')
```

Hasil histogram:

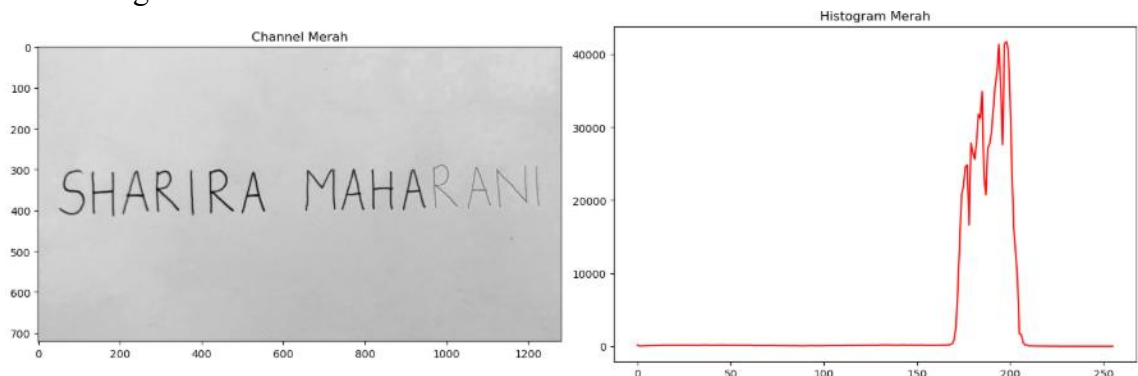


Baris pertama grid ini ditampilkan gambar asli dan histogram gabungan tiga warna. Pada histogram terlihat adanya tiga garis merah, hijau, dan biru yang saling bertumpuk. Puncak grafik berada di sisi kanan mendekati 255, hal ini membuktikan bahwa latar belakang kertas foto adalah putih bersih dan memiliki pencahayaan yang cukup.

➤ Gambar deteksi merah

```
hist_r = cv2.calcHist([r], [0], None, [256], [0, 256])
axs[1, 0].imshow(r, cmap='gray')
axs[1, 0].set_title('Channel Merah')
axs[1, 1].plot(hist_r, color='r')
axs[1, 1].set_title('Histogram Merah')
```

Hasil histogram:

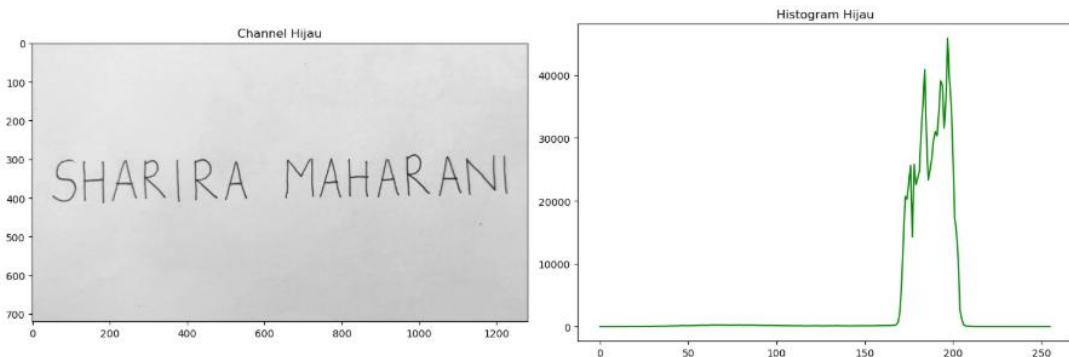


Bagian ini digunakan untuk menghitung data statistik distribusi warna merah. `cv2.calcHist` berfungsi untuk menghitung frekuensi jumlah piksel. `[r]` merupakan sumber citra yang berasal dari kanal merah hasil `splitting`. `[0]` merupakan indeks kanal, indeks 0 karena `r` hanya memiliki satu lapisan yaitu grayscale. `None` berarti yang akan dihitung adalah seluruh area gambar, bukan bagian tertentu. 256 digunakan karena citra digital 8-bit memiliki rentang nilai dari 0 hingga 255. `[0, 256]` merupakan rentang nilai yang diukur yaitu dari hitam sampai putih. Ditampilkan terlebih dahulu gambar yang diberi judul “Channel Merah” pada bagian kiri. Kemudian pada bagian kanan terdapat histogram dengan garis berwarna merah yang diberi judul “Histogram Merah”. Pada hasil histogram terlihat bahwa puncak tertinggi berada di sisi kanan, artinya ini merepresentasikan area kertas yang berwarna putih dan jumlah pikselnya sangat banyak. Sementara itu

➤ Gambar deteksi hijau

```
hist_g = cv2.calcHist([g], [0], None, [256], [0, 256])
axs[2, 0].imshow(g, cmap='gray')
axs[2, 0].set_title('Channel Hijau')
axs[2, 1].plot(hist_g, color='g')
axs[2, 1].set_title('Histogram Hijau')
```

Hasil histogram:



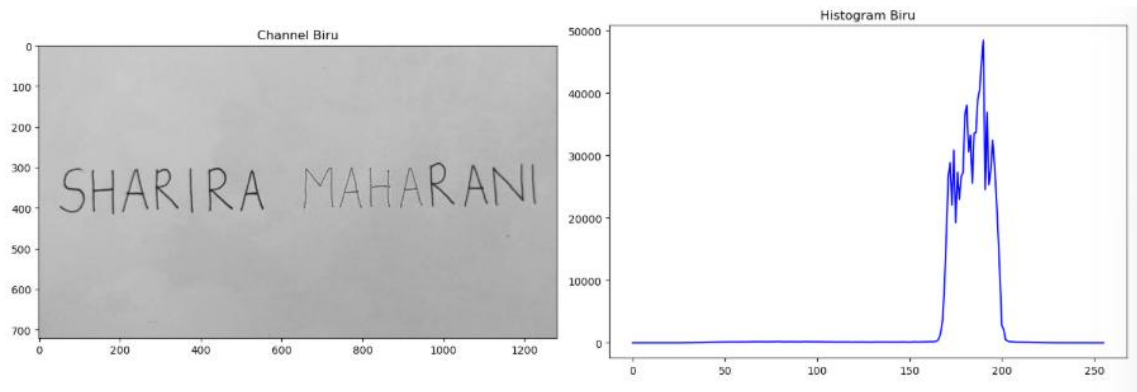
Bagian ini digunakan untuk menghitung data statistik distribusi warna hijau. `cv2.calcHist` berfungsi untuk menghitung frekuensi jumlah piksel. `[g]` merupakan sumber citra yang berasal dari kanal hijau hasil `splitting`. `[0]` merupakan indeks kanal, indeks 0 karena `g` hanya memiliki satu lapisan yaitu grayscale. `None` berarti yang akan dihitung adalah seluruh area gambar, bukan bagian tertentu. 256 digunakan karena citra digital 8-bit memiliki rentang nilai dari 0 hingga 255. `[0, 256]` merupakan rentang nilai yang diukur yaitu dari hitam sampai putih. Ditampilkan terlebih dahulu gambar yang diberi judul “Channel Hijau” pada bagian kiri. Kemudian pada bagian kanan terdapat histogram dengan garis berwarna hijau yang diberi judul “Histogram Hijau”. Pada hasil histogram terlihat bahwa puncak tertinggi juga berada di sisi kanan, artinya ini merepresentasikan area kertas yang berwarna putih dan jumlah pikselnya sangat banyak.

➤ Gambar deteksi biru

```
hist_b = cv2.calcHist([b], [0], None, [256], [0, 256])
axs[3, 0].imshow(b, cmap='gray')
axs[3, 0].set_title('Channel Biru')
axs[3, 1].plot(hist_b, color='b')
axs[3, 1].set_title('Histogram Biru')

plt.tight_layout()
plt.show()
```

Hasil histogram:



Bagian ini digunakan untuk menghitung data statistik distribusi warna terakhir yaitu biru. `cv2.calcHist` berfungsi untuk menghitung frekuensi jumlah piksel. `[b]` merupakan sumber citra yang berasal dari kanal biru hasil `splitting`. `[0]` merupakan indeks kanal, indeks 0 karena `b` hanya memiliki satu lapisan yaitu grayscale. `None` berarti yang akan dihitung adalah seluruh area gambar, bukan bagian tertentu. 256 digunakan karena citra digital 8-bit memiliki rentang nilai dari 0 hingga 255. `[0, 256]` merupakan rentang nilai yang diukur yaitu dari hitam sampai putih. Ditampilkan terlebih dahulu gambar yang diberi judul “Channel Biru” pada bagian kiri. Kemudian pada bagian kanan terdapat histogram dengan garis berwarna biru yang diberi judul “Histogram Biru”. Pada hasil histogram terlihat bahwa puncak tertinggi juga berada di sisi kanan, artinya ini merepresentasikan area kertas yang berwarna putih dan jumlah pikselnya sangat banyak.

- Thresholding

Pada bagian ini dilakukan teknik pengurangan (subtraction) untuk mendapatkan isolasi warna yang jauh lebih bersih.

- Memisahkan channel warna menjadi tiga

```
[9]:
# Memisahkan channel warna menjadi tiga
b, g, r = cv2.split(img)

fig, axs = plt.subplots(2, 2, figsize=(12, 10))
```

Pada bagian ini fungsi `cv2.split(img)` berfungsi untuk membedah citra asli menjadi tiga saluran warna dasar yaitu Blue, Green dan Red. Fungsi ini secara otomatis memecah array 3D gambar menjadi tiga matriks 2D yang berbeda. Selanjutnya, perintah `plt.subplots(2, 2)` digunakan untuk membuat grid 2 x 2, yaitu 2 baris dan 2 kolom. Pemisahan saluran warna diperlukan agar setiap komponen tinta pada nama dapat diolah secara independen sebelum dilakukan teknik pembersihan latar belakang.

- Mengisolasi bagian merah, biru dan hijau

```
# Mengisolasi bagian merah
selisih_merah = cv2.subtract(r, g)
_, hasil_merah = cv2.threshold(selisih_merah, 40, 255, cv2.THRESH_BINARY)

# Mengisolasi bagian biru
selisih_biru = cv2.subtract(b, r)
_, hasil_biru = cv2.threshold(selisih_biru, 40, 255, cv2.THRESH_BINARY)

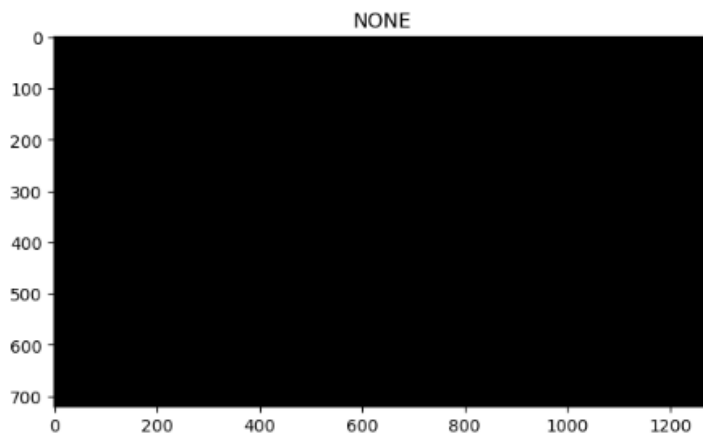
# Mengisolasi bagian hijau
selisih_hijau = cv2.subtract(g, r)
_, hasil_hijau = cv2.threshold(selisih_hijau, 18, 255, cv2.THRESH_BINARY)
```

Bagian ini menggunakan `cv2.subtract`, misalnya `r` dikurangi `g`, tujuannya adalah membuang informasi warna yang bukan merah. Dengan cara ini, hanya area yang benar-benar didominasi warna tertentu yang tersisa. Nilai ambang batas dipilih secara spesifik, 40 untuk merah dan biru karena warnanya cenderung lebih pekat, sedangkan untuk hijau dipilih nilai yang lebih sensitif yaitu 18. Hal ini dilakukan karena tinta hijau pada kertas putih seringkali memiliki kontras yang lebih tipis, sehingga angka 18 memastikan tulisan pulpen hijau tetap terdeteksi tanpa terhapus.

➤ NONE

```
# NONE (Hitam)
hitam_polos = np.zeros_like(r)
axs[0,0].imshow(hitam_polos, cmap='gray')
axs[0,0].set_title('NONE')
```

Hasil visualisasi:

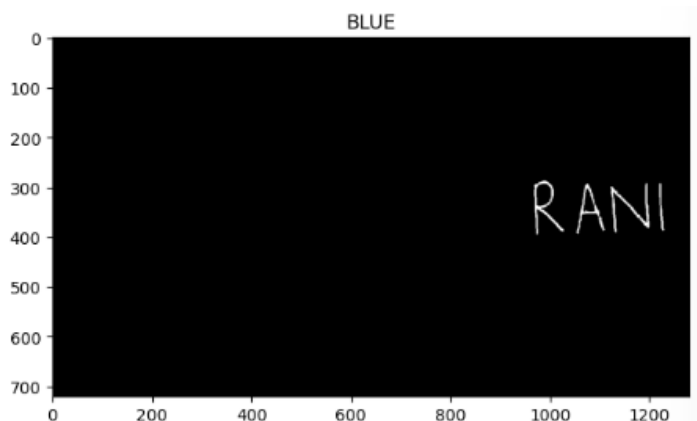


Pada bagian ini, kode `np.zeros_like(r)` berfungsi untuk menciptakan sebuah matriks kosong yang seluruh elemennya bernilai 0 yaitu hitam pekat dengan ukuran yang persis sama dengan gambar asli. Ini digunakan sebagai elemen identitas atau perbandingan awal dalam grid visualisasi. Bagian 'NONE' ini menunjukkan kondisi di mana belum ada deteksi warna yang dimasukkan ke dalam sistem.

➤ BLUE

```
# BLUE
axs[0,1].imshow(hasil_merah, cmap='gray')
axs[0,1].set_title('BLUE')
```

Hasil visualisasi:

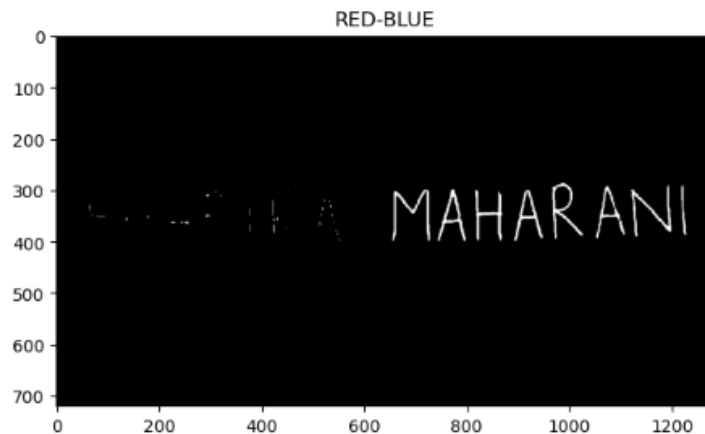


Pada bagian ini, hasil yang ditampilkan adalah hasil_merah padahal untuk warna biru. Hal ini menunjukkan hasil binerisasi dari proses isolasi warna merah yang sudah sangat bersih dari noise latar belakang kertas. Bagian yang tadinya merupakan tinta merah kini berubah menjadi warna putih terang di atas latar belakang hitam pekat. Ini membuktikan bahwa teknik pengurangan warna (subtract) yang digunakan sebelumnya berhasil memisahkan objek nama dari kertas putih secara sempurna.

➤ RED-BLUE

```
# RED-BLUE
gabung_merah_biru = cv2.bitwise_or(hasil_merah, hasil_biru)
axs[1,0].imshow(gabung_merah_biru, cmap='gray')
axs[1,0].set_title('RED-BLUE')
```

Hasil visualisasi:

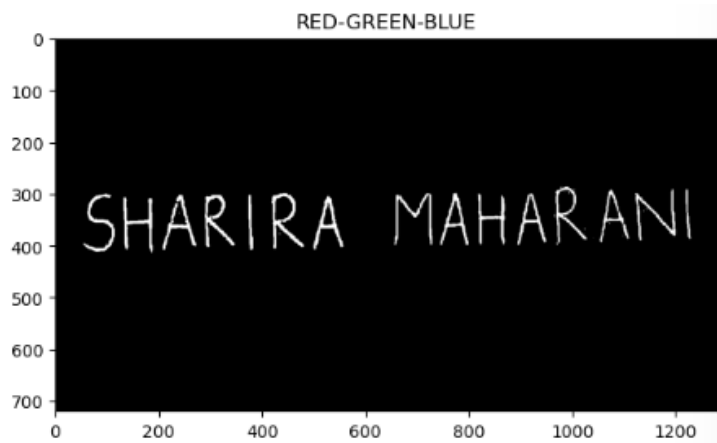


Pada bagian ini, kode menggunakan operasi logika Bitwise OR melalui `cv2.bitwise_or`. Fungsi ini menggabungkan dua citra biner (merah dan biru) menjadi satu. Logikanya adalah jika pada koordinat tertentu terdapat piksel putih di gambar merah atau di gambar biru, maka hasilnya akan tetap putih. Terlihat hasilnya adalah tulisan nama yang berwarna merah dan biru muncul bersamaan dalam satu warna putih. Teknik ini sangat efektif untuk menyatukan kembali warna yang terpisah, memastikan area nama dengan warna merah dan biru tetap utuh.

➤ RED-GREEN-BLUE

```
# RED-GREEN-BLUE
semua_warna = cv2.bitwise_or(gabung_merah_biru, hasil_hijau)
axs[1,1].imshow(semua_warna, cmap='gray')
axs[1,1].set_title('RED-GREEN-BLUE')
```

Hasil visualisasi:



Bagian akhir ini adalah menggabungkan hasil `gabung_merah_biru` dengan `hasil_hijau` menggunakan operasi Bitwise OR juga. Hasil yang didapatkan adalah citra biner lengkap yang menampilkan seluruh nama dalam tiga warna sekaligus. Analisis visual pada tahap ini menunjukkan keberhasilan total dalam meningkatkan kontras. Latar belakang kertas benar-benar hilang menjadi hitam dan semua tulisan tangan nama muncul dengan warna putih tajam. Kesimpulannya adalah sudah berhasil meningkatkan kualitas visual tanpa merusak detail citra, sehingga informasi pada nama menjadi sangat mudah terbaca.

3.3 Operasi Pixel

- Membaca gambar, konversi warna dan menampilkannya

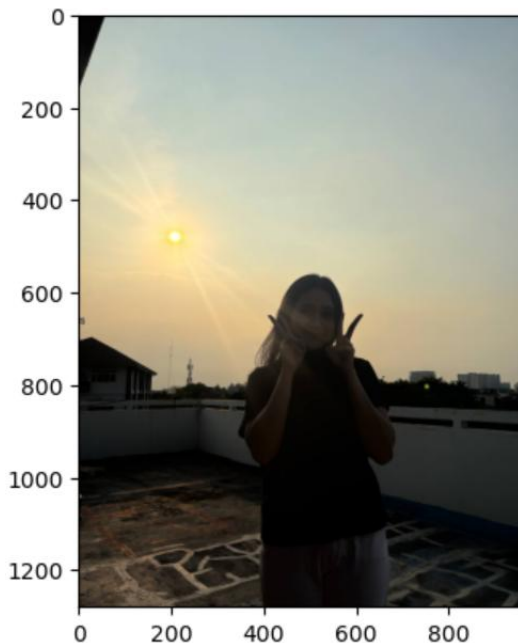
Operasi Pixel

[10]:

```
image = cv2.imread("myself.jpg")
rgb_image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
plt.imshow(rgb_image)
```

[10]:

<matplotlib.image.AxesImage at 0x26e8da547d0>



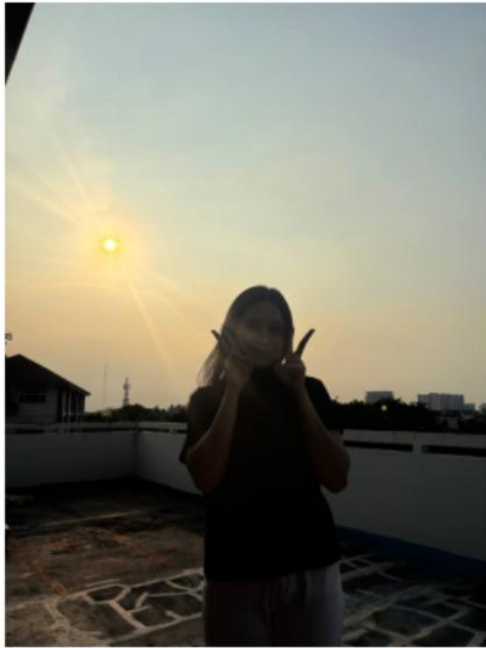
Baris kode paling awal digunakan untuk membaca file gambar baru bernama `myself.jpg` yang kemudian disimpan di variabel bernama `image`. Pada bagian ini openCV secara otomatis membaca gambar dalam format BGR (Blue-Green-Red). Selanjutnya adalah melakukan transformasi ruang warna dari format BGR bawaan OpenCV menjadi format RGB (Red-Green-Blue) menggunakan fungsi `cv2.cvtColor` yang disimpan di variabel `rgb_image`. Jika tahap ini dilewati, maka saat gambar ditampilkan, warna kulit atau pakaian pada foto akan terlihat tidak alami karena terjadi pertukaran kanal warna antara merah dan biru. Terakhir hasil gambar yang sudah di konversi tadi ditampilkan dengan menggunakan perintah `plt.show()`. Visualisasi ini akan digunakan untuk evaluasi kualitas foto pada tahap selanjutnya, sehingga nilai alpha dan beta dapat disesuaikan.

- Menampilkan gambar asli

[11]:

```
# Menampilkan Gambar Asli
plt.imshow(rgb_image)
plt.title("Gambar Asli")
plt.axis("off")
plt.show()
```

Gambar Asli



Bagian ini sama dengan langkah sebelumnya yaitu menampilkan gambar asli yang sudah di konversi. Kemudian diberi judul “Gambar Asli”, pada hasil plot terlihat bahwa sumbu x dan y tidak dimunculkan karena kode `plt.axis("off")`. Gambar ditampilkan dengan menggunakan `plt.show()`.

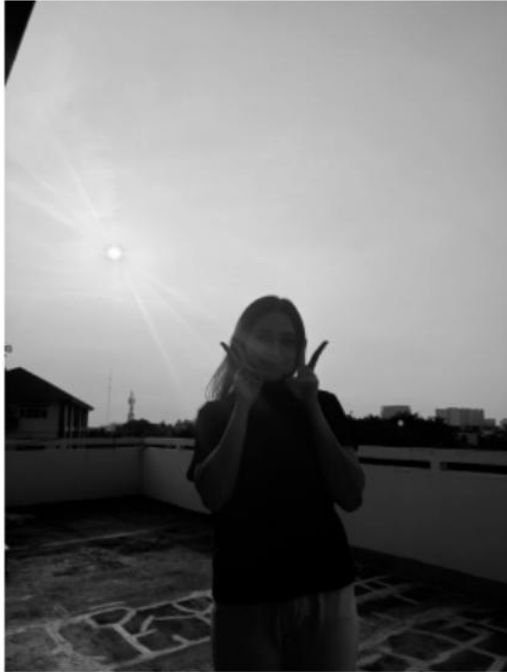
- Menampilkan gambar gray

[12]:

```
# Menampilkan Gambar Abu
gambar_gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

plt.imshow(gambar_gray, cmap = 'gray')
plt.title("Gambar Gray")
plt.axis("off")
plt.show()
```

Gambar Gray



Baris kode pertama berfungsi untuk menyederhanakan citra dengan mengubah format warna BGR menjadi grayscale yang hanya mempunyai 1 kanal yaitu abu-abu. Secara teknis, proses ini menggunakan rumus bobot intensitas cahaya (luminance) untuk menggabungkan ketiga kanal warna menjadi satu nilai intensitas tunggal per piksel. Selanjutnya hasil ditampilkan menggunakan `plt.imshow()`. Perintah `cmap="gray"` digunakan agar gambar ditampilkan dengan palet warna hitam putih. Perintah `plt.axis("off")` digunakan untuk menghilangkan koordinat sumbu. Berbeda dengan gambar asli yang memiliki informasi warna (merah, hijau, biru), gambar gray hanya menyisakan informasi intensitas cahaya. Pada gambar gray, dapat terlihat dengan lebih jelas bahwa area profil memiliki tingkat keabuan yang berbeda dengan latar belakang. Ini adalah titik awal untuk menentukan nilai alpha dan beta nantinya. Dibandingkan gambar asli, gambar gray lebih menonjolkan tekstur dan detail garis.

- Menampilkan gambar gray yang dicerahkan

[13]:

```
# Membuat Gambar Abu yang dicerahkan
[baris, kolom] = gambar_gray.shape[:2]

beta = 16.5
gambar_cerah = np.zeros((baris, kolom, 3))

for x in range(baris):
    for y in range(kolom):
        gyx = gambar_gray[x,y] + beta
        gambar_cerah[x,y] = gyx

gambar_cerah = gambar_cerah.astype(np.uint8)

plt.imshow(gambar_cerah, cmap='gray')
plt.title("Gambar Gray yang Dicerahkan")
plt.axis("off")
plt.show()
```

Gambar Gray yang Dicerahkan



Bagian ini menggunakan teknik point processing untuk manipulasi kecerahan citra secara manual. Baris pertama berfungsi untuk mengambil dimensi resolusi gambar agar bisa melakukan iterasi yang sesuai. Variabel $\beta = 16.5$ dideklarasikan sebagai nilai penambah kecerahan. Selanjutnya, `np.zeros` digunakan untuk menyiapkan kanvas kosong dengan ukuran yang sama namun memiliki 3 kanal warna. Penggunaan $\beta = 16.5$ dipilih untuk meningkatkan intensitas cahaya secara halus agar area profil tetap terlihat natural tanpa kehilangan detail akibat cahaya yang terlalu ekstrem.

Selanjutnya digunakan nested loop untuk melakukan penelusuran piksel demi piksel. Pada setiap koordinat (x, y) , nilai intensitas abu-abu asli ditambah dengan nilai β . Secara matematis, operasi ini menggeser seluruh histogram ke arah kanan atau area terang. Proses manual ini menunjukkan bagaimana setiap titik pada foto dimanipulasi secara individual untuk mencapai efek pencahayaan yang diinginkan.

Karena hasil penjumlahan sebelumnya bisa menghasilkan angka desimal, kode `.astype(np.uint8)` digunakan untuk mengembalikan tipe data ke integer 8-bit (0-255) agar dapat dibaca kembali sebagai citra standar. Kemudian hasil gambar ditampilkan menggunakan `plt.imshow()` dengan judul “Gambar Gray yang dicerahkan”.

Dibandingkan dengan gambar grayscale asli, gambar hasil pencerahan ini terlihat memiliki titik putih yang lebih dominan. Latar belakang yang tadinya abu-abu kusam kini menjadi sedikit lebih terang, mendekati putih. Dengan penambahan β , detail pada area wajah dan pakaian yang tadinya gelap menjadi lebih terlihat. Secara teknis, gambar ini memiliki warna putih yang cukup banyak, sehingga untuk proses pengaturan kontras atau α , rentang nilai piksel sudah berada di area yang lebih optimal untuk dipertajam.

- Menampilkan gambar gray yang diperkontras

```
[14]:  
  
# Membuat Gambar abu yang di kontraskan  
alpha = 1.06  
gambar_kontras = np.zeros((baris, kolom, 3))  
  
for x in range(baris):  
    for y in range(kolom):  
        gyx = gambar_gray[x,y] * alpha  
        gambar_kontras[x,y] = gyx  
  
gambar_kontras = gambar_kontras.astype(np.uint8)  
  
plt.imshow(gambar_kontras, cmap='gray')  
plt.title("Gambar Gray yang Diperkontras")  
plt.axis("off")  
plt.show()
```

Gambar Gray yang Diperkontras



Bagian ini juga merupakan proses perubahan kontras citra secara manual. Pada bagian pertama variabel α dideklarasikan dengan nilai 1.06, α berfungsi sebagai konstanta pengali intensitas yang merepresentasikan tingkat kontras. Nilai 1.06 dipilih untuk meningkatkan kontras secara halus. Selanjutnya, disiapkan matriks `gambar_kontras` menggunakan `np.zeros` sebagai wadah kosong untuk menampung hasil perhitungan piksel yang baru.

Selanjutnya terdapat nested loop yang digunakan untuk menjalankan operasi point processing setiap piksel pada koordinat (x,y) . Berbeda dengan pencerahan yang menggunakan penjumlahan, kontras menggunakan perkalian yaitu setiap nilai intensitas abu-abu dikalikan dengan α . Proses ini akan merenggangkan rentang dinamis pada histogram, dimana nilai yang terang akan menjadi jauh lebih terang, sedangkan nilai yang gelap perubahannya tidak terlalu drastis.

Hasil perkalian sering kali menghasilkan angka desimal atau nilai yang melebihi rentang 255, sehingga `.astype(np.uint8)` digunakan untuk memastikan data kembali ke format 8-bit yang sesuai. Selanjutnya gambar ditampilkan menggunakan `plt.imshow()` dengan palet warna abu-abu.

Dibandingkan dengan gambar asli atau gambar gray biasa, gambar yang diperkontras ini memiliki perbedaan yang lebih tajam antara area gelap seperti bayangan dan rambut dengan area terang. Objek terlihat lebih berdimensi dan tidak datar. Nilai $\alpha = 1.06$ berhasil meningkatkan kontras di area yang gelap secara efektif. Area profil sekarang tampak lebih menonjol karena batas-batas tepian objek menjadi lebih tegas dibandingkan latar belakang yang terang.

Pemilihan nilai 1.06 menunjukkan kehati-hatian dalam menghindari efek color burn. Karena pengalinya tidak terlalu besar, detail pada area yang sudah terang tidak hilang menjadi putih polos, sehingga kualitas visual citra tetap terjaga sesuai batas toleransi yang diminta. Jika pada tahap pencerahan seluruh gambar menjadi lebih terang secara merata, pada tahap kontras ini gambar menjadi lebih jelas dalam membedakan antara intensitas rendah dan tinggi,

- Menampilkan gambar gray yang dicerahkan dan dikontraskan

```
[15]:
# Membuat gambar abu yang dipencerah dan dikontraskan
beta = 15
alpha = 1
gambar_hasil = np.zeros((baris, kolom, 3))

for x in range(baris):
    for y in range(kolom):
        gyx = alpha * gambar_gray[x,y] + beta
        gambar_hasil[x,y] = gyx

gambar_hasil = gambar_hasil.astype(np.uint8)

plt.imshow(gambar_hasil, cmap='gray')
plt.title("Gambar Gray Dicerahkan dan Dikontraskan")
plt.axis("off")
plt.show()
```

Gambar Gray Dicerahkan dan Dikontraskan



Bagian ini adalah tahap akhir yaitu menggabungkan dua teknik transformasi linear sekaligus untuk mendapatkan hasil yang paling optimal. Pertama-tama ditetapkan dua parameter alpha dan beta. Nilai beta = 15 berfungsi sebagai komponen pencerahan yang akan meningkatkan nilai intensitas cahaya secara keseluruhan. Nilai alpha = 1 berfungsi sebagai pengali kontras. Meskipun angka 1 berarti kontras tetap, namun secara struktur nilai ini sudah siap untuk melakukan peregangan kontras jika nilai alpha diubah di atas 1. Variabel gambar_hasil disiapkan sebagai matriks kosong untuk menampung hasil akhir pengolahan.

Pada bagian nested loop terdapat rumus yang merupakan tahapan perhitungan utama. Setiap piksel citra asli dengan alpha terlebih dahulu untuk mengatur rentang kontrasnya, kemudian ditambah dengan beta untuk mengatur tingkat kecerahannya. Kode `.astype(np.uint8)` memastikan bahwa hasil perhitungan matematis kembali ke format citra 8-bit yang standar. Hasil visual yang ditampilkan merupakan akumulasi dari seluruh proses sebelumnya.

Dibandingkan dengan gambar gray asli yang cenderung redup, hasil gabungan ini menunjukkan peningkatan kualitas yang signifikan. Area profil terlihat lebih terang karena nilai beta, sementara detail garis wajah dan pakaian tetap sama. Penggunaan alpha = 1 dan beta = 15 memberikan hasil yang seimbang. Gambar tidak terlihat terlalu ekstrem atau pecah, namun tetap mencapai target utama yaitu membuat area profil lebih menonjol dibandingkan latar belakang yang terang.

Analisis visual menunjukkan bahwa tidak terjadi efek color burn yang berlebihan. Karena nilai alpha tidak dipaksakan tinggi, detail-detail halus pada foto tetap terlihat jelas dan tidak menjadi putih polos. Perbandingan antara gambar asli dan hasil akhir ini membuktikan efektivitas program dalam mengolah citra secara manual, objek utama kini memiliki kejelasan yang lebih baik.

- Menampilkan semua gambar

[16]:

```
# Menampilkan semua hasil gambar

fig, axs = plt.subplots(5, figsize=(10,20))
axs[0].imshow(rgb_image)
axs[0].set_title('Gambar Asli')

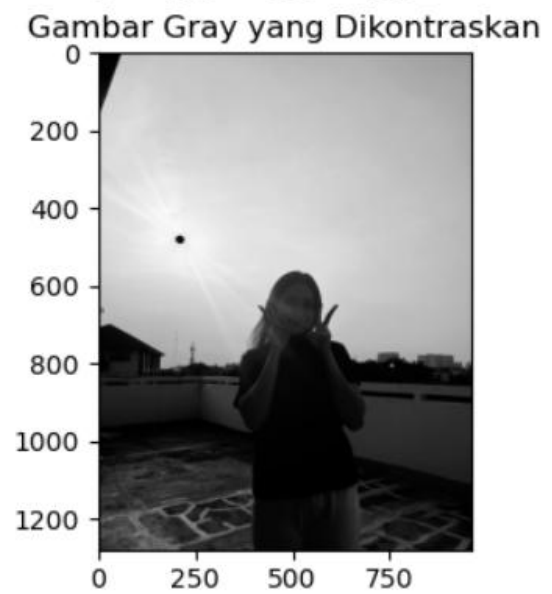
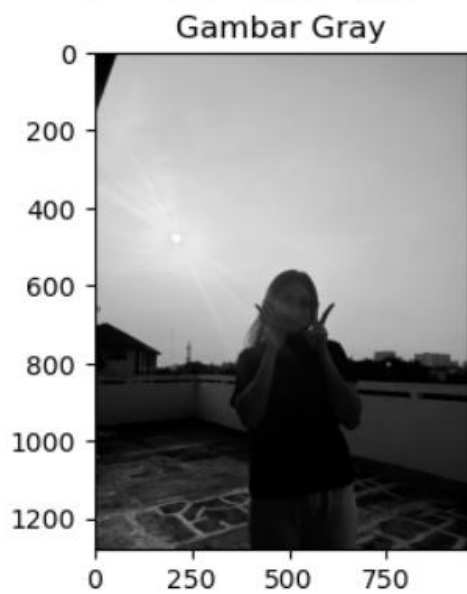
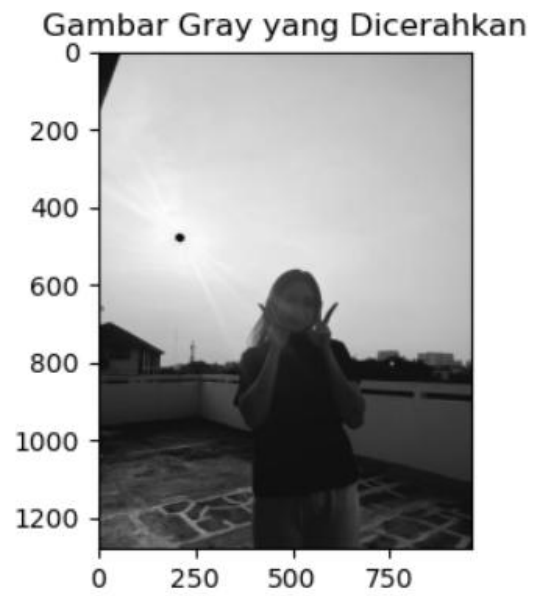
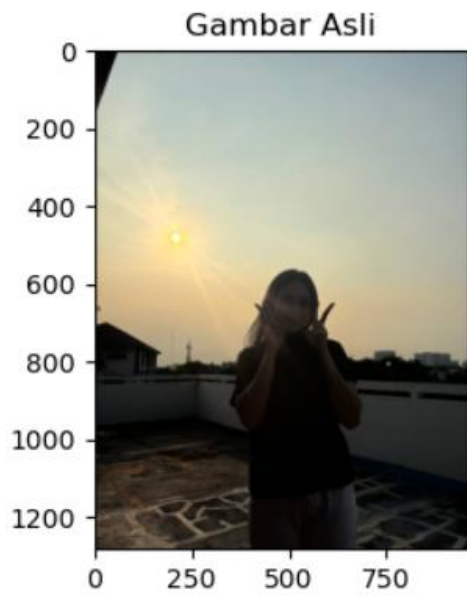
axs[1].imshow(gambar_gray, cmap='gray')
axs[1].set_title('Gambar Gray')

axs[2].imshow(gambar_cerah, cmap='gray')
axs[2].set_title('Gambar Gray yang Dicerahkan')

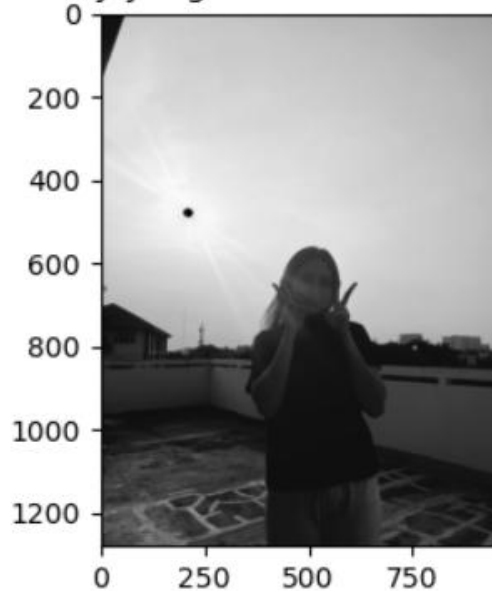
axs[3].imshow(gambar_kontras, cmap='gray')
axs[3].set_title('Gambar Gray yang Dikontraskan')

axs[4].imshow(gambar_hasil, cmap='gray')
axs[4].set_title('Gambar Gray yang Dicerahkan dan Dikontraskan')
```

Bagian kode ini berfungsi sebagai tahap akhir untuk memvisualisasikan seluruh rangkaian transformasi citra secara sistematis dalam satu bingkai grid menggunakan fungsi `plt.subplots(5)`. Dengan mengatur ukuran gambar melalui `figsize=(10,20)`, plot menyusun lima tahapan citra secara vertikal memanjang kebawah sesuai dengan urutan, mulai dari gambar asli hingga gambar yang sudah dicerahkan dan dikontraskan.



Gambar Gray yang Dicerahkan dan Dikontraskan



BAB IV

PENUTUP

Berdasarkan landasan teori dan analisis hasil praktikum yang telah dilakukan dapat ditarik beberapa kesimpulan utama sebagai berikut:

1. Keberhasilan Ekstraksi dan Deteksi Warna

Proses pemisahan kanal warna (splitting channel) menggunakan library OpenCV dan NumPy terbukti efektif dalam mengisolasi komponen warna Merah, Hijau, dan Biru dari citra asli. Penggunaan teknik pengurangan citra (image subtraction) dan operasi logika Bitwise OR memungkinkan pendeteksian objek tulisan tangan secara spesifik dan bersih, sehingga setiap warna tinta dapat diidentifikasi secara terpisah maupun digabungkan kembali tanpa gangguan latar belakang.

2. Peranan Vital Histogram dalam Analisis Citra

Histogram berfungsi sebagai instrumen statistik yang sangat akurat untuk memahami karakteristik pencahayaan dan kontras citra. Melalui distribusi frekuensi piksel, praktikan dapat menentukan kondisi citra (apakah terlalu gelap atau terang) dan menetapkan nilai ambang batas (threshold) yang objektif. Puncak histogram pada area intensitas tinggi secara konsisten merepresentasikan latar belakang kertas putih, sementara sebaran nilai pada intensitas rendah merepresentasikan objek tulisan dan profil.

3. Efektivitas Nilai Ambang Batas (Thresholding)

Pemilihan nilai ambang batas yang berbeda untuk setiap kanal warna (seperti nilai 40 untuk merah/biru dan 18 untuk hijau) merupakan langkah krusial dalam binerisasi. Hal ini membuktikan bahwa setiap kanal warna memiliki sensitivitas yang berbeda terhadap cahaya. Penggunaan nilai ambang batas yang presisi berhasil memisahkan objek dari latar belakang dengan tingkat kontras yang tinggi, meminimalisir noise, dan menjaga integritas bentuk tulisan tangan nama mahasiswa.

4. Optimalisasi Kualitas Citra melalui Parameter Alpha dan Beta

Transformasi linear menggunakan parameter Alpha (Kontras) dan Beta (Brightness) terbukti secara signifikan dapat meningkatkan kualitas visual citra. Penambahan nilai Beta berhasil mengangkat intensitas cahaya pada area yang redup, sementara peningkatan nilai Alpha mempertegas perbedaan antara objek profil dan latar belakang. Hasil akhir menunjukkan bahwa kombinasi kedua parameter ini mampu membuat area profil mahasiswa menjadi lebih menonjol dan jelas sesuai instruksi soal, dengan tetap menjaga batas toleransi agar tidak terjadi efek color burn yang merusak detail citra.

5. Relevansi Teori dan Praktik

Seluruh tahapan praktikum yang dilakukan selaras dengan landasan teori mengenai Digital Image Processing. Implementasi kode Python tidak hanya sekadar manipulasi visual, tetapi merupakan penerapan nyata dari konsep matriks piksel, transformasi ruang warna, dan pemrosesan titik (point processing) yang bertujuan untuk menghasilkan informasi digital yang lebih bermakna dan terstruktur.

DAFTAR PUSTAKA

- Untuk, C., Tanda, D., Di, T., Prayoga, F., & Gibran, M. K. (2025). *Penerapan Teknik Binarisasi Dan Segmentasi*. 14, 623–630.
- Renaldo, E., Pratama, M. F. R., & Prasetya, F. (2022). *OPERASI TITIK PADA PENGOLAHAN CITRA DIGITAL UNTUK MATLAB*. 200–205.
- Suhada, I., Ermawati, A., Paris, M., Maulidhan, R., & Rosyani, P. (2024). *Konversi Citra RGB Ke Grayscale Menggunakan OpenCV*. 2(3), 390–394.
- Ashillah, S., Gulo, S. A., Rahmi, Y., & Niska, D. Y. (2025). *Implementasi Algoritma Pengolahan Citra Digital untuk Perbaikan Kualitas Gambar*.
- Riwanto, Y., Yanuar, I., Pratiwi, R., & Fikri, M. A. (2021). *Segmentasi Citra Menggunakan Pendekatan Trial and Error dan Metode Otsu untuk Identifikasi Objek*. 371–378.
- Novanni, W. D., Sukma, L., Nufus, R., Ramadhan, M. S., Informatika, T., Darma, U. B., Plaju, K., Palembang, K., Sobel, O., Berwarna, C., & Grayscale, C. (2025). *DAN GRAYSCALE DENGAN OPERATOR SOBEL*. 9(4), 7264–7269.